

## ادامه جزوه دوم: مرور و اکسچنج پخش

این فایل شامل ادامه گفت‌وگو پس از جزوه دوم است: نکات اصلاح شده در مرور و مراحل ۱ تا ۹ آموزش اکسچنج پخش. آموزش نوع موضوعی هنوز شروع نشده است. کدها برای نسخه ۷ کتابخانه نوشته شده‌اند RabbitMQ.Client.

### بخش اول — نکات تثبیت شده در مرور

#### ۱. برگشت به تولیدکننده یا برگشت به صف

هنگام انتشار با `mandatory: true`، اگر هیچ صف منطقی نباشد، پیام به تولیدکننده برمی‌گردد و رویداد `Nack` با `requeue` اجرا می‌شود. اما بعد از دریافت پیام توسط مصرف‌کننده، دستور `BasicReturnAsync` پیام را به همان صفی برمی‌گرداند که از آن دریافت شده بود `true`.

منظور از صف اصلی در این مثال، صف اعلان است؛ پیام به تولیدکننده بر نمی‌گردد. مصرف‌کننده فعلی یا مصرف‌کننده دیگری روی همان صف می‌تواند آن را دوباره دریافت کند. اگر مصرف‌کننده خاموش باشد ولی صف متصل باشد، پیام داخل صف منتظر می‌ماند.

#### ۲. پیام تکراری و مسئول تشخیص آن

هنگام ارسال مجدد همان پیام، مقدار `MessageId` حفظ می‌شود تا مصرف‌کننده بتواند تکرار را تشخیص دهد. بروکر صرفاً با دیدن شناسه یکسان پیام را حذف نمی‌کند. مصرف‌کننده با بررسی جدول `Inbox` عملیات تکراری را اجرا نمی‌کند و پیام را به بروکر تأیید می‌کند.

ثبت شناسه و تغییر موجودی باید در یک تراکنش باشند؛ هر دو با هم ثبت یا لغو می‌شوند. قید یکتا روی کرش می‌تواند باعث `Ack` و قبل از `Commit` مانع ثبت هم‌زمان شناسه یکسان می‌شود. بعد از `MessageId` تحویل مجدد شود؛ بررسی `Inbox` مانع افزایش دوباره موجودی است.

#### ۳. خطای دیتابیس و تلاش مجدد

قطع اتصال دیتابیس نشانه تکراری بودن پیام نیست. نباید آن را موفق فرض کرد و `Ack` فرستاد. بعد از دریافت مجدد، ابتدا شناسه را بررسی می‌کنیم؛ ممکن است تراکنش قبلی ثبت شده باشد ولی نتیجه‌اش به برنامه نرسیده باشد. برای قطعی ادامه‌دار، تلاش مجدد با فاصله و تعداد محدود مناسب است.

#### ۴. مقصد پیام‌های مرده، ساخت اکسچنج نیست

گزینه `x-dead-letter-exchange` فقط نام اکسچنج مقصد را روی صف مشخص می‌کند؛ خودش اکسچنج نمی‌سازد. گزینه `x-dead-letter-routing-key` کلید انتشار پیام مرده در آن مقصد را تعیین می‌کند. ساخت اکسچنج جداگانه با `ExchangeDeclareAsync` انجام می‌شود.

با `Nack` و `requeue: false`، در صورت تنظیم درست مسیر، پیام از طریق اکسچنج مقصد به صف شکست‌خورده‌ها می‌رود. بدون تنظیم مسیر حذف می‌شود. صف شکست‌خورده‌ها پیام را اصلاح نمی‌کند؛ علت شکست باید بررسی شود.

## ۵. صف انتظار و کلید اختصاصی

زمان اعتبار 5000 میلی‌ثانیه یعنی ۵ ثانیه. پیام منقضی‌شده به اکسچنج تنظیم‌شده می‌رود؛ مقصد نهایی الزاماً صف شکست‌خورده‌ها نیست. در مسیر تلاش مجدد، مقصد را برای بازگشت به صف اعلان تنظیم کردیم.

صف انتظار مصرف‌کننده ندارد؛ وگرنه ممکن است پیام قبل از پایان زمان انتظار دریافت شود. کلید می‌توانست پیام تلاش payment.success فقط به صف اعلان متصل است. استفاده از notification.retry مجدد را به گزارش‌گیری هم بفرستد. اضافه‌کردن اتصال جدید، اتصال قبلی صف را حذف نمی‌کند.

## ۶. مرز پایان اکسچنج مستقیم

مسیریابی مستقیم، تطابق دقیق کلیدها، دریافت پیام توسط چند صف و مدیریت پیام بدون مسیر تمام شده بود. انتقال امن پیام خطا دار به صف انتظار و محدود کردن تعداد تلاش‌ها هنوز پیاده‌سازی نشده‌اند. این‌ها ادامه قابلیت اطمینان هستند، نه شرط فهم مسیریابی مستقیم.

## بخش دوم — آموزش اکسچنج پخش

### مرحله ۱ — مفهوم اکسچنج پخش

در نوع Fanout، پیام به تمام صف‌های متصل به اکسچنج فرستاده می‌شود. مقدار Routing Key در انتخاب صف‌ها نقشی ندارد. اگر اعلان، گزارش‌گیری و حسابداری هرکدام صف متصل داشته باشند، هر سه یک نسخه از پیام می‌گیرند. در نوع مستقیم، تطابق دقیق کلید لازم است؛ در نوع پخش، اتصال صف کافی است.

### مرحله ۲ — ساخت اکسچنج و اتصال صف

```
await channel.ExchangeDeclareAsync(
    exchange: "payment.fanout",
    type: ExchangeType.Fanout,
    durable: true,
    autoDelete: false,
    arguments: null);

await channel.QueueBindAsync(
    queue: "payment.notification.queue",
    exchange: "payment.fanout",
    routingKey: "");
```

پارامتر type با مقدار ExchangeType.Fanout نوع پخش را تعیین می‌کند. نام جدید انتخاب شده چون نوع اکسچنج موجود payment.direct را نمی‌توان با اعلام مجدد تغییر داد. در اتصال بالا فرض کرده‌ایم صف قبلاً ساخته شده است. کلید را خالی می‌گذاریم چون در این نوع نادیده گرفته می‌شود.

### مرحله ۳ — اتصال چند صف

```
await channel.QueueBindAsync(
    queue: "payment.reporting.queue",
    exchange: "payment.fanout",
    routingKey: "");
```

با اتصال صف گزارش‌گیری نیز، هر دو صف یک نسخه مستقل می‌گیرند. تأیید پیام در صف اعلان نسخه گزارش‌گیری را حذف نمی‌کند. هر سرویس مستقل پردازش و تأیید می‌کند.

### مرحله ۴ — انتشار پیام

```
var body = Encoding.UTF8.GetBytes("Payment succeeded");

await channel.BasicPublishAsync(
    exchange: "payment.fanout",
    routingKey: "",
    mandatory: true,
    basicProperties: new BasicProperties(),
    body: body);
```

پیام به تمام صف‌های متصل می‌رسد. حتی اگر کلید را payment.failed بگذاریم، مقصدها تغییر نمی‌کنند. در این مثال ساده، اگر هیچ صفی متصل نباشد و مسیر جایگزین هم نداشته باشیم، mandatory باعث برگشت پیام به تولیدکننده می‌شود؛ برای مدیریت آن همان رویداد برگشت مطالب قبلی لازم است.

## مرحله ۵ — چند صف در برابر چند مصرف‌کننده

اکسچنج پخش‌ی پیام را بین صف‌ها تکثیر می‌کند، نه میان تمام مصرف‌کننده‌های یک صف. اگر دو مصرف‌کننده روی یک صف باشند، در تحویل معمول هر پیام به یکی از آن‌ها می‌رسد. پیام‌های صف میان مصرف‌کننده‌ها تقسیم می‌شوند؛ تحویل مجدد در شرایط شکست همچنان ممکن است.

در مثال دو صف، اعلان یک نسخه می‌گیرد و یکی از دو مصرف‌کننده اعلان آن را دریافت می‌کند. گزارش‌گیری نیز نسخه خودش را به مصرف‌کننده گزارش می‌دهد. بنابراین یک انتشار به دو تحویل معمول، در دو صف متفاوت، منجر می‌شود.

## مرحله ۶ — صف مستقل برای هر سرویس

برای دریافت مستقل همه رویدادها توسط حسابداری، یک صف جدید می‌سازیم و به اکسچنج متصل می‌کنیم. اتصال مصرف‌کننده حسابداری به صف اعلان باعث رقابت بر سر پیام‌ها می‌شود، نه دریافت نسخه مستقل. اگر سرویس گزارش‌گیری خاموش باشد، پیام‌ها می‌توانند در صف متصل آن منتظر بمانند و سرویس اعلان پردازش را ادامه دهد. فرض این مثال برقرارماندن صف و نبودن انقضا یا حذف پیام است.

## مرحله ۷ — اتصال دیرهنگام صف

اکسچنج پیام‌ها را برای صف‌های آینده نگه نمی‌دارد. اگر صف حسابداری پس از انتشار پیام متصل شود، آن پیام قبلی را دریافت نمی‌کند؛ فقط انتشارهای بعد از اتصال به آن مسیریابی می‌شوند. خاموش‌بودن مصرف‌کننده با نبودن اتصال صف متفاوت است.

## مرحله ۸ — انتخاب نوع اکسچنج

اگر همه صف‌های متصل باید تمام پیام‌ها را بگیرند، نوع پخش‌ی مناسب است. اگر هر صف باید فقط کلید مشخص خودش را دریافت کند، نوع مستقیم مناسب‌تر است. در اکسچنج پخش‌ی، انتشار پرداخت موفق و ناموفق هر دو به تمام صف‌های متصل می‌رسد.

## مرحله ۹ — جمع‌بندی و تمرین نهایی

نکته اصلی: تکثیر بین صف‌هاست؛ تقسیم کار بین مصرف‌کننده‌های یک صف است. کلید مسیریابی مقصدهای پخش‌ی را تغییر نمی‌دهد. صف تازه‌متصل، پیام گذشته را دریافت نمی‌کند. مفهوم اصلی این نوع در چت کامل شد؛ نوع موضوعی فقط معرفی شد و هنوز آموزش داده نشده است.

## بخش سوم — پرسش و پاسخ‌های آموزش پخشی

پرسش: سه صف متصل‌اند؛ کدام‌ها پیام را می‌گیرند؟

پاسخ: هر سه صف متصل به همان اکسچنج.

پرسش: کدام پارامتر نوع پخشی را تعیین می‌کند؟

پاسخ: پارامتر type با مقدار ExchangeType.Fanout.

پرسش: چرا کلید اتصال را خالی گذاشتیم؟

پاسخ: چون نوع پخشی کلید را برای انتخاب صف به کار نمی‌برد.

پرسش: آیا Ack صف اعلان نسخه گزارش‌گیری را حذف می‌کند؟

پاسخ: خیر؛ نسخه‌های صف‌ها مستقل‌اند.

پرسش: اگر کلید انتشار payment.failed باشد، فقط صف خطا می‌گیرد؟

پاسخ: خیر؛ همچنان تمام صف‌های متصل پیام را می‌گیرند.

پرسش: آیا دو مصرف‌کننده روی یک صف، هر دو نسخه هر پیام را می‌گیرند؟

پاسخ: خیر؛ در تحویل معمول یکی از آن‌ها هر پیام را می‌گیرد.

پرسش: برای حسابداری مستقل، صف جدید یا مصرف‌کننده روی اعلان؟

پاسخ: یک صف جداگانه متصل به اکسچنج.

پرسش: آیا صف بعداً متصل‌شده پیام قبلی را می‌گیرد؟

پاسخ: خیر؛ اکسچنج سابقه پیام‌ها را برای آن بازیخس نمی‌کند.

پرسش: برای انتخاب مقصد با کلید دقیق چه نوعی مناسب است؟

پاسخ: اکسچنج مستقیم.

پرسش: دو صف داریم؛ اعلان دو مصرف‌کننده و گزارش یکی. نتیجه یک انتشار؟

پاسخ: دو صف نسخه می‌گیرند؛ در حالت معمول دو مصرف‌کننده دریافت می‌کنند، یکی از اعلان و یکی از گزارش.

## مرورهای بعدی

طبق درخواست، تکرار موارد اشتباه برای جلسه دیگری گذاشته شد. موارد نیازمند مرور: تفاوت mandatory و requeue، مسئول تشخیص تکرار، تفاوت تنظیم مقصد با ساخت اکسچنج، و نقش صف بدون مصرف‌کننده در تأخیر. مطالب جدید بعدی می‌تواند اکسچنج موضوعی باشد.